Info:
The code first initializes by calibrating the color sensor to base values without light to better recognize different color values in comparison to a state without light being shown to the lens of the receiver's body. The photoresistor within the receiver apparatus helps detect when light is being shown. After going through multiple resistors, light counts as being lit onto the lens if the photoresistor generates a value greater than 12. After calibration, the program sets maximum and minimum frequencies to calculate sound from, before beginning to read in color from an outside source. The values read from the photoresistor are then clamped into a desired color range as the values read may not be in a readily readable state, before these new mapped values are put into an RGB array. Finally, the 3 RGB values in the array are converted into a singular hue float that is used to determine what color is being detected and what sound needs to be played. The hue is then scaled to a tone based on the high and low note constants, and that tone is played if the photoresistor detects enough light.

```
// Pin assignments
const int S0 = 4;
const int S1 = 7;
const int S2 = 8;
const int S3 = 12;
const int OUT = 2;
const int speaker = A4;
const int photor = A0;


//calibratedRGB values set rgb range after calibration. here we set them to an
arbitrarily "small" value, and set the loop counter to 0.
int calibratedR = -99999999;
int calibratedG = -99999999;
int calibratedB = -99999999;
int calibrationCounter = 0;


//Note range constants
const float NOTE_LOW = 262; // 262 // 523
const float NOTE_HIGH = 1046;


//Buffer for hue wrap handling
const int BUFFER = 15;


//RGB range for determining if no note should be playing
const int MIN_RGB = 9;
const int MAX_RGB = 215;
```

```
// int lastLEDTime = 0;




void setup() {


 //Set pins. everything is output from arduino except the OUT pin of the color sensor
and the photoresistor
 pinMode(S0, OUTPUT);
 pinMode(S1, OUTPUT);
 pinMode(S2, OUTPUT);
 pinMode(S3, OUTPUT);
 pinMode(OUT, INPUT);
 pinMode(speaker, OUTPUT);
 pinMode(photor, INPUT);

 // pinMode(LED_BUILTIN, OUTPUT);


 // Set frequency scaling to 100%
 digitalWrite(S0, HIGH);
 digitalWrite(S1, HIGH);


 //start the serial monitor
 Serial.begin(9600);
 Serial.println("Program Start");
 Serial.println(' ');

 // lastLEDTime = millis();


}
```

```
void loop() {
  //Testing photoresistor
 // if (millis() - lastLEDTime >= 250) {
 //   digitalWrite(LED_BUILTIN, !digitalRead(LED_BUILTIN));
 //   lastLEDTime = millis();
 // }


 int pr = analogRead(A0);
 Serial.print("Photoresistor value: ");
 Serial.println(pr);


 //Read color sensor values, print them to monitor
 int redIn = readColor(LOW, LOW);
 int greenIn = readColor(HIGH, HIGH);
 int blueIn = readColor(LOW, HIGH);


 Serial.print("Raw red value: ");
 Serial.println(redIn);
 Serial.print("Raw green value: ");
 Serial.println(greenIn);
 Serial.print("Raw blue value: ");
 Serial.println(blueIn);
 Serial.println(" ");


 //Calibrate the sensor
 if(calibrationCounter < 4) {
   calibrationCounter++;
   if(redIn > calibratedR)
     calibratedR = redIn;
   if(greenIn > calibratedG)
     calibratedG = greenIn;
   if(blueIn > calibratedB)
     calibratedB = blueIn;


   if(calibrationCounter == 4) {
     tone(speaker, 261, 125);
     delay(125);
```

```
    tone(speaker, 289, 125);
    delay(125);
    tone(speaker, 577, 250);
    delay(250);


    Serial.println("Calibrated!");
    Serial.print("Calibrated R: ");
    Serial.println(calibratedR);
    Serial.print("Calibrated G: ");
    Serial.println(calibratedG);
    Serial.print("Calibrated B: ");
    Serial.println(calibratedB);
    Serial.println(" ");
    Serial.println("NEW DATA");
    Serial.println("--------");
    Serial.println(" ");
  }
  return;
}


//create array with RGB values
int cIn[3] =
{
  clampmap(redIn, 0, calibratedR, 255, 0),       //red
  clampmap(greenIn, 0, calibratedG, 255, 0),     //green
  clampmap(blueIn, 0, calibratedB, 255, 0)       //blue
};


Serial.print("Mapped red value: ");
Serial.println(cIn[0]);
Serial.print("Mapped green value: ");
Serial.println(cIn[1]);
Serial.print("Mapped blue value: ");
Serial.println(cIn[2]);
Serial.println(" ");


//create hue variable and cal RGBToHue, offset it to accomadate red being on both
ends of the scale
```

```
 int hue = RGBToHue(cIn);
 Serial.print("Hue before wrap handling: ");
 Serial.println(hue);




 if(hue > 360 - BUFFER) {
   hue = 0;
 } else if(hue > 360 - (2 * BUFFER)) {
   hue = 360 - (2 * BUFFER);
 }


 Serial.print("Hue after wrap handling: ");
 Serial.println(hue);
 Serial.println(' ');


 //Map hue to a range of tones
 int outFreq = map(hue, 0, 360 - (2 * BUFFER), NOTE_LOW, NOTE_HIGH);
 Serial.print("Calculated tone: ");
 Serial.println(outFreq);
 Serial.println(' ');
 Serial.println("NEW DATA");
 Serial.println("--------");
 Serial.println(" ");


 //play a note if you see light within a certain range
//   if(ShouldTurnOff(cIn)) {
//     Serial.println("No note");
//     noTone(speaker);
//   } else {
//     tone(speaker, outFreq);
//   }


 //play a note
 if (pr > 12) {
   tone(speaker, outFreq);
 }
```

```
 else {
   noTone(speaker);
 }
}









 //Establish function that reads the filter states of the color sensor to map those
values
int readColor(int s2State, int s3State) {
 digitalWrite(S2, s2State);
 digitalWrite(S3, s3State);
 // delay(500); // settle time, no longer using, but this is where it was
 return pulseIn(OUT, digitalRead(OUT) == HIGH ? LOW : HIGH);
}
 //Establish function that clamps input values into desired range
int clampmap(int in, int min1, int max1, int min2, int max2) {
 if(min1 < max1) {    //if calibrated is positive
   if(in < min1)          //if input is negative
     in = min1;             //input = 0
   if(in > max1)          //if input is larger than calibrated max
     in = max1;             //input = calibrated max
 } else {             //if calibrated is not positive
   if(in > min1)          //if input is positive
     in = min1;             //input = 0
   if(in < max1)          //if input is smaller then calibrated max
     in = max1;             //input = calibrated max
 }
 return map(in, min1, max1, min2, max2);
}


//Establish function that converts RGB values into a single hue value
```

```
int RGBToHue(int iRGB[]) {
 float fRGB[3] = { iRGB[0] / 255.0f, iRGB[1] / 255.0f, iRGB[2] / 255.0f };
 int max = fRGB[0] > fRGB[1] ? 0 : 1;
 max = fRGB[max] > fRGB[2] ? max : 2;
 int min = fRGB[0] < fRGB[1] ? 0 : 1;
 min = fRGB[min] < fRGB[2] ? min : 2;


 if(max == min)
   return 0;


 float h = (fRGB[(max + 1) % 3] - fRGB[(max + 2) % 3]) / (fRGB[max] - fRGB[min]);


 if(max == 0) {
   h = (int)h % 6;
 } else {
   h += 2 * max;
 }


 h = (int)((h * 60) + 0.5f);


 return (((int)h % 360) + (h < 0 ? 360 : 0)) % 360;
}


//Establishes function to determine if input values are within range that it should
not be playing notes in
// bool ShouldTurnOff(int iRGB[]) {
//   if(iRGB[0] < MIN_RGB && iRGB[1] < MIN_RGB && iRGB[2] < MIN_RGB)
//     return true;
//   if(iRGB[0] > MAX_RGB && iRGB[1] > MAX_RGB && iRGB[2] > MAX_RGB)
//     return true;
//   return false;
// }
```